

HW2 任务二：/generate 请求源码流程图与说明

流程图（文本版）

POST /generate

```
-> TokenizerManager.generate_request [manage
rs/tokenizer_manager.py]
-> tokenize -> Req -> ZMQ -> Scheduler
-> event_loop_normal / waiting_queue [manage
rs/scheduler.py]
-> get_new_batch_prefill [manage
rs/schedule_policy.py]
-> match_prefix_for_req -> RadixCache.match_pr
efix [mem_cache/radix_cache.py]
    (命中前缀 -> 复用 KV; 未命中 -> 需 Prefill)
-> PrefillAdder / ScheduleBatch [model_
executor/schedule_batch.py]
-> TpModelWorker: Prefill(首token) -> Decode(自回
归) [managers/tp_worker.py]
-> cache_finished_req / cache_unfinished_req
    [mem_cache/radix_cache.py]
-> DetokenizerManager -> TokenizerManager stre
aming output -> Client
```

说明与问答

SGLang v0.5.14 /generate 请求主流程追踪

流程总览（对应流程图）

1. **请求接收与排队**：FastAPI 收到 POST /generate，`TokenizerManager.generate\_request()`（python/sglang/srt/entrypoints/http_server.py 调用；实现在 python/sglang/srt/managers/tokenizer_manager.py）把文本/ input_ids tokenize 成 Req，经 ZMQ 发给 Scheduler，进入 `schedule.r.waiting\_queue`。
2. **调度与前缀匹配**：Scheduler 事件循环 `event\_loop\_normal()`（python/sglang/srt/managers/scheduler.py）每轮调用 `get\_new\_batch\_prefill()`（schedule_policy.py 中 PrefillAdder），对每个 Req 执行 `match\_prefix\_for\_req()` -> `RadixCache.match\_prefix()`（python/sglang/srt/mem_cache/radix_cache.py），沿 radix 树按 token 前缀匹配已缓存的 KV，命中部分标记 prefix_indices，未命中部分才需要 Prefill。
3. **Prefill**：`ScheduleBatch`（model_executor/schedule_batch.py）合并后发给 `TpModelWorker`（managers/tp_worker.py），对未命中 token 做一次前向计算，产出首个输出 token 与新 KV。
4. **Decode**：请求进入 running batch，`event\_loop\_normal` 每步对整批做一次 decode 前向，自回归生成直到 EOS 或 max_new_tokens。
5. **缓存写回**：请求完成后 `RadixCache.cache\_finished\_req()`（未完成批次用 `cache\_unfinished\_req()`）把新产生的 KV 按 token 前缀插入 radix 树，供后续请求复用；LRU 淘汰。
6. **流式返回**：TokenizerManager 的 stream_out 循环把每个新

token 封装为 JSON 行 (data: {...}) 经 HTTP chunked 推回客户端；`meta\_info.finish\_reason` 终止。

关键问答

- **如何进入等待队列**：TokenizerManager token 化后经 ZMQ socket 发送，Scheduler 主循环 `recv_requests()` 后 append 到 `waiting_queue`，按 `fcfs/lpm` 等策略被 `PrefillAdder` 取出组批。
- **前缀匹配如何减少 Prefill token**：RadixCache 以 token 序列建基数树，`match_prefix` 返回最长可复用前缀的 KV 索引；请求只需对后缀做 Prefill（本次实验 shared 组 512 token 中 256 命中，Prefill 减半）。
- **新 KV 如何写回**：Prefill/Decode 产出的 KV 位于 token pool；请求结束（或每次 chunk）时 `cache_finished_req/cache_unfinished_req` 以 token 序列为键把 KV 页插入 radix 树，引用计数管理生命周期。
- **流式返回**：DetokenizerManager 增量反解码，TokenizerManager 按生成顺序逐 token 通过 `asyncio queue` 写回 HTTP 流式响应。